

Python-Based Malware Detection Tool

- **Objective:** To develop a malware scanner that detects malicious files using **hash-based detection**.

Or

And also Improving Cs_Week03, Branch Day03, Task03.

Here are the code where you can make changes to improve the malware scanner:

```
malware_scanner.py

File Edit View

import os
import hashlib
import requests
import pefile
import threading
import logging

# Setup logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

# Sample database of known malware hashes
known_malware_hashes = {
    "a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9": "example MD5 malware hash",
    "d1d2d3d4d5d6d7d8d9d0d1d2d3d4d5": "example SHA1 hash",
    "000f8bc04e1d373eade4e832e7b4f8": "example SHA256 hash"
}

# Suspicious API Functions used in malware
suspicious_imports = ("CreateRemoteThread", "VirtualAlloc", "LoadLibrary")

def calculate_hashes(file_path):
    """Compute SHA256, SHA1, and MD5 hashes of a file."""
    hashes = {'sha256': hashlib.sha256(), 'sha1': hashlib.sha1(), 'md5': hashlib.md5()}

    try:
        with open(file_path, 'rb') as f:
            while chunk := f.read(4096):
                for algo in hashes.values():
                    algo.update(chunk)
            return {name: algo.hexdigest() for name, algo in hashes.items()}
    except Exception as e:
        logging.error(f"Error computing hashes for {file_path}: {e}")
        return None

def check_hash_in_database(file_hashes):
    """Check if a file hash is in the known malware database."""
    for hash_value in file_hashes.values():
        if hash_value in known_malware_hashes:
            return f"⚠️ WARNING: {hash_value} is a known malware!"
    return "✅ File hash is clean."

def check_executable(file_path):
    """Check if a file is an executable based on its extension."""
    return file_path.lower().endswith((".exe", ".dll", ".sys"))

def check_suspicious_functions(file_path):
    """Analyze an executable for suspicious API calls."""
    try:
        pe = pefile.pe(file_path)
        imports = []
        if hasattr(pe, "DIRECTORY_ENTRY_IMPORT"):
            for entry in pe.DIRECTORY_ENTRY_IMPORT:
                imports.extend([imp.name.decode() for imp in entry.imports if imp.name])

        suspicious_found = [func for func in suspicious_imports if func in imports]
        return f"⚠️ Suspicious functions detected: {', '.join(suspicious_found)}" if suspicious_found else "✅ No suspicious functions detected."
    except Exception:
        return "⚠️ Error analyzing the file."

def check_virustotal(file_hash):
    """Check File Hash Against Virustotal."""
    VIRUSTOTAL_API_KEY = "76a890"  # Replace with actual API Key
    if VIRUSTOTAL_API_KEY == "76a890":
        return "⚠️ Virustotal check skipped (API Key missing)."

    url = f"https://www.virustotal.com/api/v3/files/{file_hash}"
    headers = {"x-apikey": VIRUSTOTAL_API_KEY}

    try:
        response = requests.get(url, headers=headers)
        if response.status_code == 200:
            data = response.json()
            detections = data.get("data", {}).get("attributes", {}).get("last_analysis_stats", {}).get("malicious", 0)
            return f"⚠️ {detections} detections on virustotal." if detections > 0 else "✅ No detections on virustotal."
        else:
            return f"⚠️ Virustotal query failed: {response.text}"
    except requests.RequestException as e:
        return f"⚠️ Virustotal request error: {e}"

def scan_file(file_path):
    """Scan an individual file for malware."""
    logging.info(f"Scanning: {file_path}")
    hashes = calculate_hashes(file_path)
    if not hashes:
        return

    # Check against known malware database
    logging.info(check_hash_in_database(hashes))

    # Check if executable and scan
    if check_executable(file_path):
        logging.info(check_suspicious_functions(file_path))

    # Query Virustotal
    logging.info(check_virustotal(hashes['sha256']))

def scan_directory(directory_path):
    """Scan all files in a directory for malware."""
    if not os.path.isdir(directory_path):
        logging.error("Invalid directory path.")
        return

    logging.info(f"🔍 Scanning directory: {directory_path}")

    threads = []
    for root, _, files in os.walk(directory_path):
        for file in files:
            file_path = os.path.join(root, file)
            t = threading.Thread(target=scan_file, args=(file_path,))
            t.start()
            threads.append(t)

    # Wait for all threads to complete
    for t in threads:
        t.join()

    logging.info("✅ Scan complete.")

if __name__ == "__main__":
    target_directory = input("Enter the directory path to scan: ")
    scan_directory(target_directory)
```

1. Known Malware Hash Database:-

- Update with more real malware hashes from a trusted database.

2. VirusTotal API Integration

- Without an API key, VirusTotal scanning won't work.

3. Suspicious Function Detection

- This will improve detection of potentially harmful files.

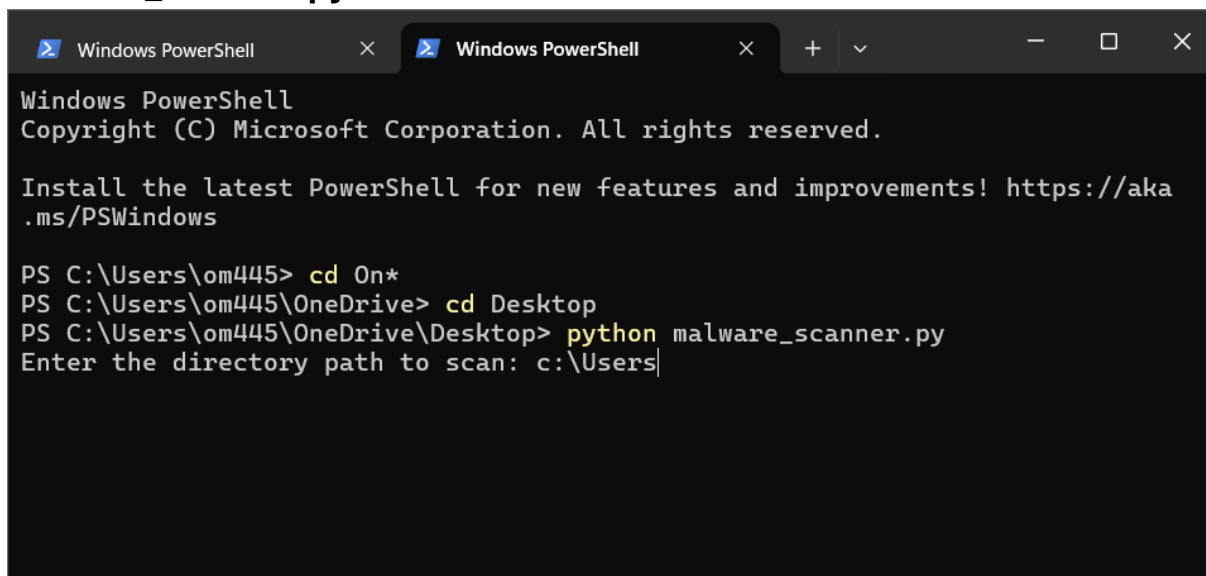
4. Multithreading Optimization

- Daemon threads ensure the script exits cleanly.

5. Logging & Output Format

- This improves readability and debugging.

malware_scanner.py is saved and run:

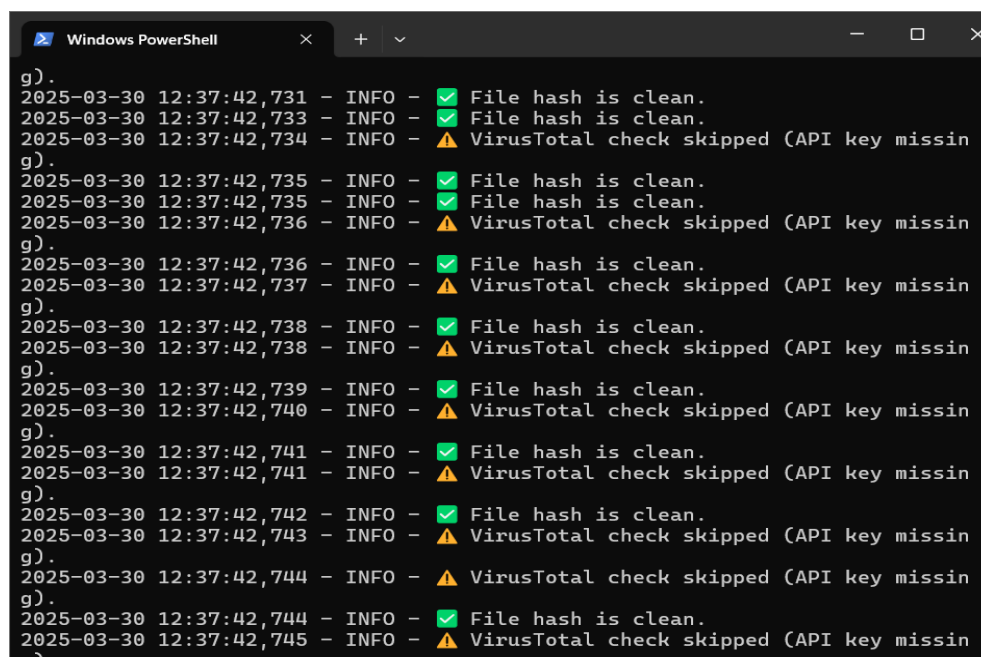


```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\om445> cd On*
PS C:\Users\om445\OneDrive> cd Desktop
PS C:\Users\om445\OneDrive\Desktop> python malware_scanner.py
Enter the directory path to scan: c:\Users\
```

After changing output:



```
g).
2025-03-30 12:37:42,731 - INFO - ✓ File hash is clean.
2025-03-30 12:37:42,733 - INFO - ✓ File hash is clean.
2025-03-30 12:37:42,734 - INFO - ⚠ VirusTotal check skipped (API key missin
g).
2025-03-30 12:37:42,735 - INFO - ✓ File hash is clean.
2025-03-30 12:37:42,735 - INFO - ✓ File hash is clean.
2025-03-30 12:37:42,736 - INFO - ⚠ VirusTotal check skipped (API key missin
g).
2025-03-30 12:37:42,736 - INFO - ✓ File hash is clean.
2025-03-30 12:37:42,737 - INFO - ⚠ VirusTotal check skipped (API key missin
g).
2025-03-30 12:37:42,738 - INFO - ✓ File hash is clean.
2025-03-30 12:37:42,738 - INFO - ⚠ VirusTotal check skipped (API key missin
g).
2025-03-30 12:37:42,739 - INFO - ✓ File hash is clean.
2025-03-30 12:37:42,740 - INFO - ⚠ VirusTotal check skipped (API key missin
g).
2025-03-30 12:37:42,741 - INFO - ✓ File hash is clean.
2025-03-30 12:37:42,741 - INFO - ⚠ VirusTotal check skipped (API key missin
g).
2025-03-30 12:37:42,742 - INFO - ✓ File hash is clean.
2025-03-30 12:37:42,743 - INFO - ⚠ VirusTotal check skipped (API key missin
g).
2025-03-30 12:37:42,744 - INFO - ⚠ VirusTotal check skipped (API key missin
g).
2025-03-30 12:37:42,744 - INFO - ✓ File hash is clean.
2025-03-30 12:37:42,745 - INFO - ⚠ VirusTotal check skipped (API key missin
g).
```